

```
# Module 1: Understanding complex datasets
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
```

```
housingData=pd.read_csv("demos.csv")
```

```
housingData.shape
```

```
(1460, 81)
```

```
housingData.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1460 entries, 0 to 1459
Data columns (total 81 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Id                     1460 non-null   int64
1   MSSubClass             1460 non-null   int64
2   MSZoning               1460 non-null   object
3   LotFrontage           1201 non-null   float64
4   LotArea               1460 non-null   int64
5   Street                1460 non-null   object
6   Alley                 91 non-null     object
7   LotShape              1460 non-null   object
8   LandContour           1460 non-null   object
9   Utilities             1460 non-null   object
10  LotConfig             1460 non-null   object
11  LandSlope             1460 non-null   object
12  Neighborhood          1460 non-null   object
13  Condition1            1460 non-null   object
14  Condition2            1460 non-null   object
15  BldgType              1460 non-null   object
16  HouseStyle            1460 non-null   object
17  OverallQual           1460 non-null   int64
18  OverallCond           1460 non-null   int64
19  YearBuilt             1460 non-null   int64
20  YearRemodAdd          1460 non-null   int64
21  RoofStyle            1460 non-null   object
22  RoofMatl             1460 non-null   object
23  Exterior1st          1460 non-null   object
24  Exterior2nd          1460 non-null   object
25  MasVnrType           588 non-null    object
26  MasVnrArea           1452 non-null   float64
27  ExterQual            1460 non-null   object
28  ExterCond            1460 non-null   object
29  Foundation           1460 non-null   object
30  BsmtQual            1423 non-null   object
31  BsmtCond            1423 non-null   object
32  BsmtExposure        1422 non-null   object
33  BsmtFinType1        1423 non-null   object
34  BsmtFinSF1          1460 non-null   int64
35  BsmtFinType2        1422 non-null   object
36  BsmtFinSF2          1460 non-null   int64
37  BsmtUnfSF           1460 non-null   int64
38  TotalBsmtSF          1460 non-null   int64
39  Heating             1460 non-null   object
40  HeatingQC           1460 non-null   object
41  CentralAir           1460 non-null   object
42  Electrical           1459 non-null   object
43  1stFlrSF            1460 non-null   int64
44  2ndFlrSF            1460 non-null   int64
```

```

45 LowQualFinSF      1460 non-null    int64
46 GrLivArea         1460 non-null    int64
47 BsmtFullBath      1460 non-null    int64
48 BsmtHalfBath      1460 non-null    int64
49 FullBath          1460 non-null    int64
50 HalfBath           1460 non-null    int64
51 BedroomAbvGr      1460 non-null    int64

```

```

housingData.describe()
# append ['GrLivArea'] to view only that column

```

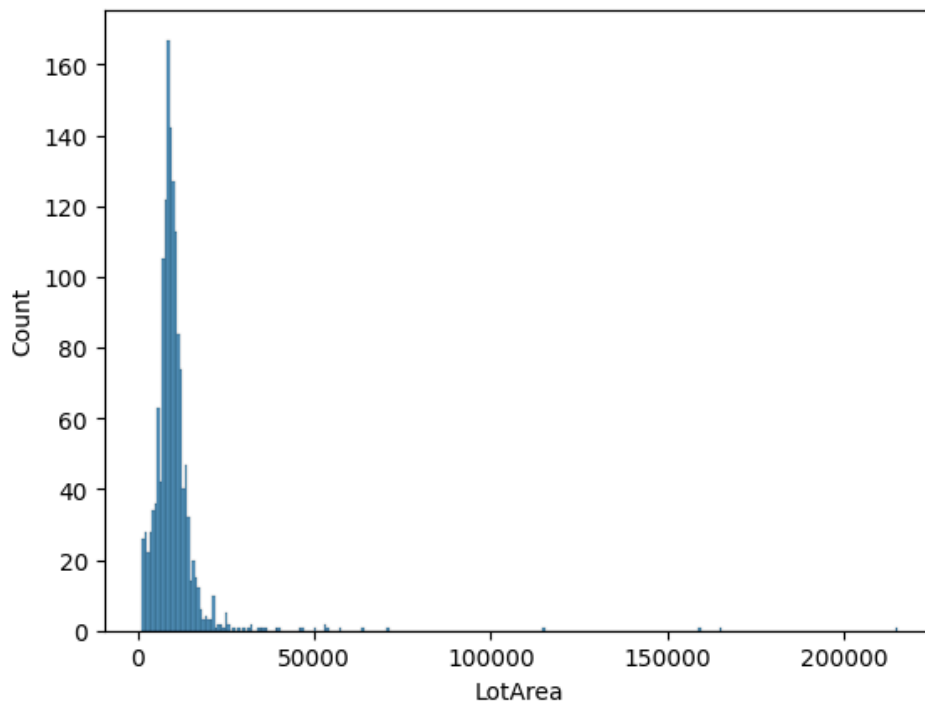
|              | <b>Id</b>   | <b>MSSubClass</b> | <b>LotFrontage</b> | <b>LotArea</b> | <b>OverallQual</b> | <b>OverallCond</b> | <b>YearBuilt</b> |
|--------------|-------------|-------------------|--------------------|----------------|--------------------|--------------------|------------------|
| <b>count</b> | 1460.000000 | 1460.000000       | 1201.000000        | 1460.000000    | 1460.000000        | 1460.000000        | 1460.000000      |
| <b>mean</b>  | 730.500000  | 56.897260         | 70.049958          | 10516.828082   | 6.099315           | 5.575342           | 1971.267800      |
| <b>std</b>   | 421.610009  | 42.300571         | 24.284752          | 9981.264932    | 1.382997           | 1.112799           | 30.202900        |
| <b>min</b>   | 1.000000    | 20.000000         | 21.000000          | 1300.000000    | 1.000000           | 1.000000           | 1872.000000      |
| <b>25%</b>   | 365.750000  | 20.000000         | 59.000000          | 7553.500000    | 5.000000           | 5.000000           | 1954.000000      |
| <b>50%</b>   | 730.500000  | 50.000000         | 69.000000          | 9478.500000    | 6.000000           | 5.000000           | 1973.000000      |
| <b>75%</b>   | 1095.250000 | 70.000000         | 80.000000          | 11601.500000   | 7.000000           | 6.000000           | 2000.000000      |
| <b>max</b>   | 1460.000000 | 190.000000        | 313.000000         | 215245.000000  | 10.000000          | 9.000000           | 2010.000000      |

8 rows × 38 columns

```
# Visualization
```

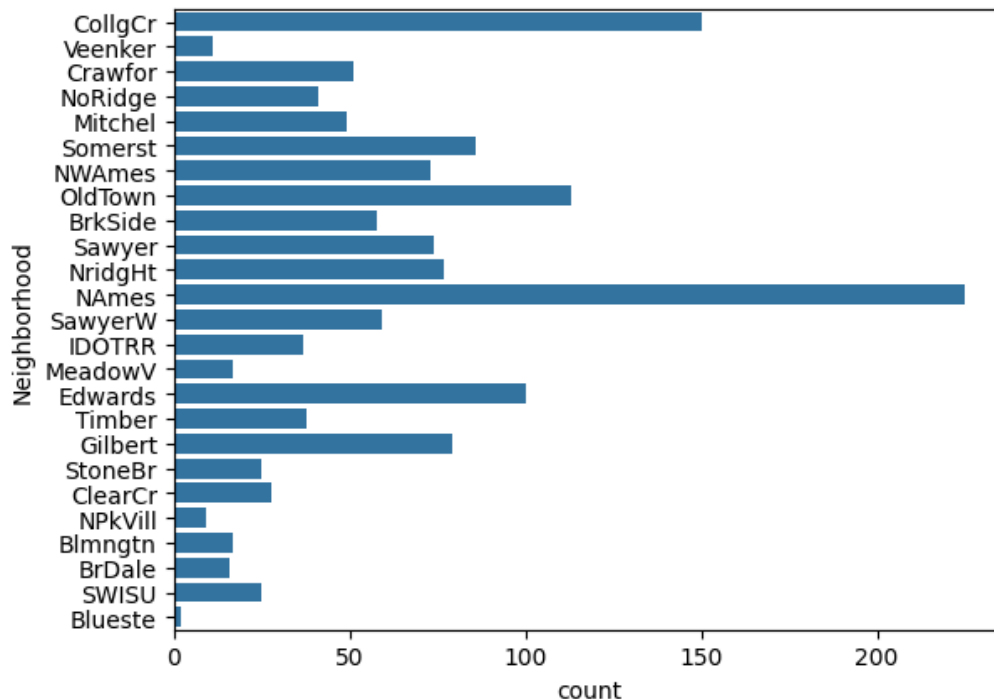
```
# plotting a histogram of the LotArea feature can show us if most homes fall within a certain
sns.histplot(housingData['LotArea'])
```

<Axes: xlabel='LotArea', ylabel='Count'>



```
# Next, let's take a look at count plots, which are great for visualizing categorical features
sns.countplot(housingData['Neighborhood'])
```

<Axes: xlabel='count', ylabel='Neighborhood'>



```
# missing values
missing_values = housingData.isnull()
```

missing\_values

|      | Id    | MSSubClass | MSZoning | LotFrontage | LotArea | Street | Alley | LotShape | LandContour | U   |
|------|-------|------------|----------|-------------|---------|--------|-------|----------|-------------|-----|
| 0    | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 1    | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 2    | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 3    | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 4    | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| ...  | ...   | ...        | ...      | ...         | ...     | ...    | ...   | ...      | ...         | ... |
| 1455 | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 1456 | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 1457 | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 1458 | False | False      | False    | False       | False   | False  | True  | False    | False       |     |
| 1459 | False | False      | False    | False       | False   | False  | True  | False    | False       |     |

1460 rows × 81 columns

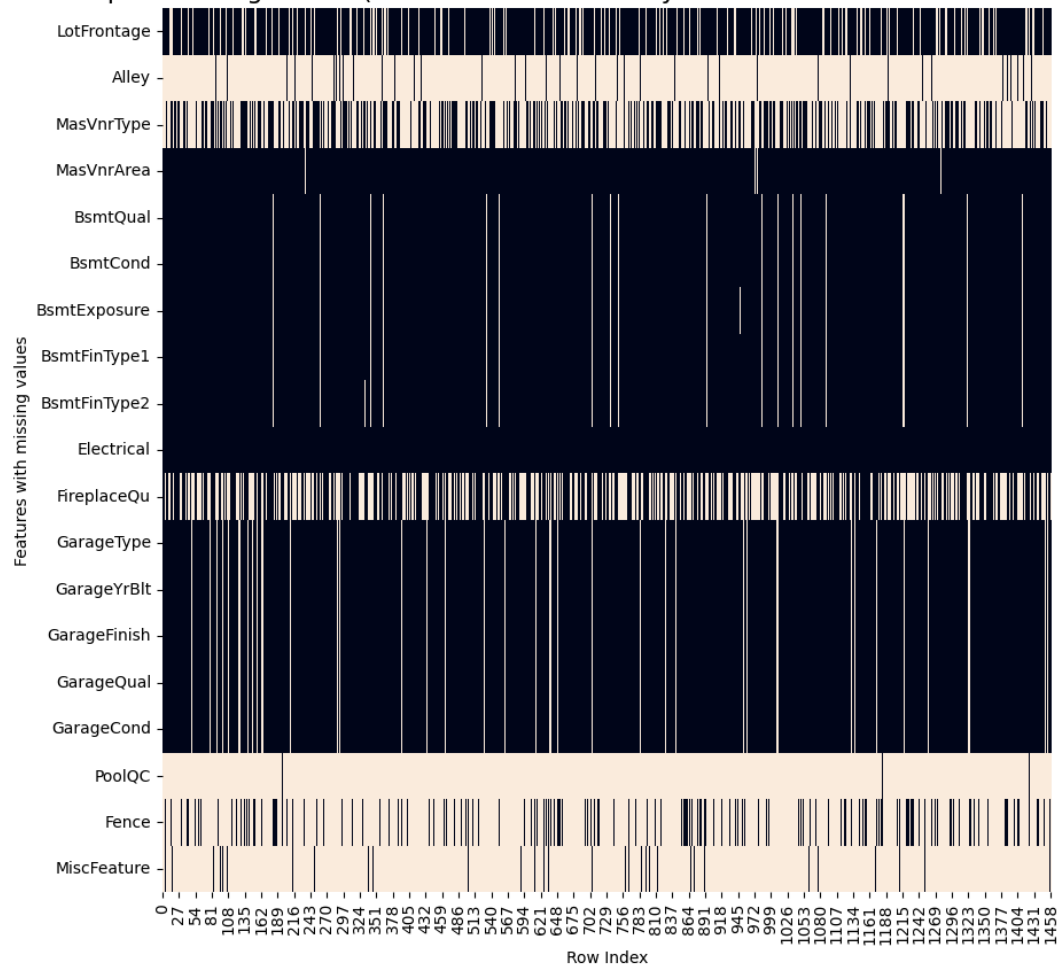
```
# filter only the columns with at least one missing value
missingCols = housingData.columns[housingData.isnull().any()]
missingDataSubset = housingData[missingCols]
```

```
# plot the same using matplotlib and seaborn
plt.figure(figsize=(10,10))
sns.heatmap(missingDataSubset.isnull().T, cbar=False)

plt.title("Heatmap of missing values (Columns with nulls only - skin color are nulls which we
```

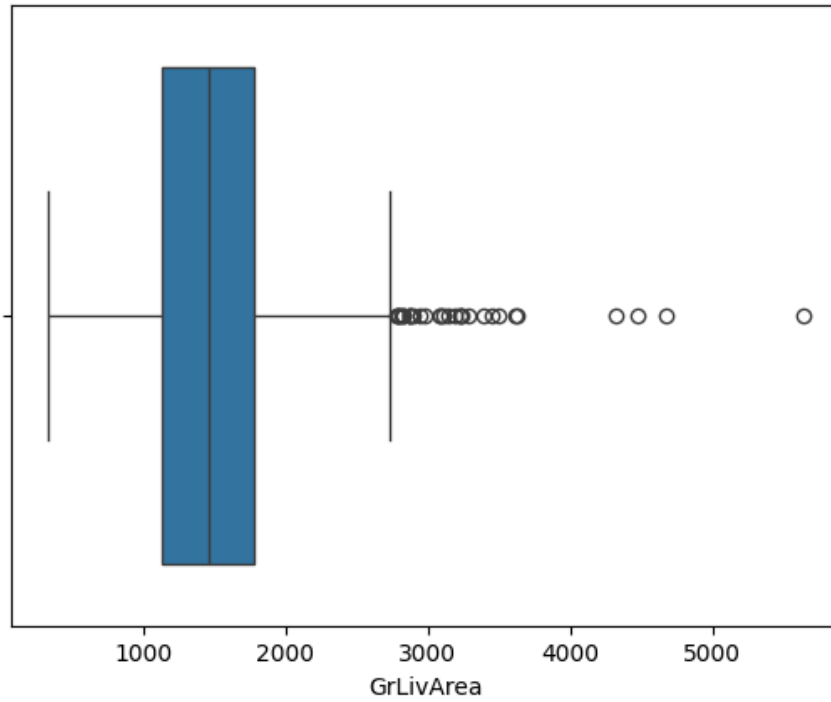
```
plt.xlabel("Row Index")
plt.ylabel("Features with missing values")
plt.xticks(rotation=0)
plt.show()
```

Heatmap of missing values (Columns with nulls only - skin color are nulls which we will clean)



```
sns.boxplot(x=housingData['GrLivArea'])
```

<Axes: xlabel='GrLivArea'>



```
# Module 2: Cleaning Complex data for EDA
```

```
# Scaling and dimentionality reduction
```

```
# Additional Libraries to import  
from sklearn.preprocessing import StandardScaler  
from sklearn.decomposition import PCA
```

```
housingDataClean = housingData.copy()
```

```
missingValues = housingDataClean.isnull().sum().sort_values(ascending=False)
```

```
missingValues[missingValues > 0]
```

|                     | $\emptyset$ |
|---------------------|-------------|
| <b>PoolQC</b>       | 1453        |
| <b>MiscFeature</b>  | 1406        |
| <b>Alley</b>        | 1369        |
| <b>Fence</b>        | 1179        |
| <b>MasVnrType</b>   | 872         |
| <b>FireplaceQu</b>  | 690         |
| <b>LotFrontage</b>  | 259         |
| <b>GarageQual</b>   | 81          |
| <b>GarageFinish</b> | 81          |
| <b>GarageType</b>   | 81          |
| <b>GarageYrBlt</b>  | 81          |
| <b>GarageCond</b>   | 81          |
| <b>BsmtFinType2</b> | 38          |
| <b>BsmtExposure</b> | 38          |
| <b>BsmtCond</b>     | 37          |
| <b>BsmtQual</b>     | 37          |
| <b>BsmtFinType1</b> | 37          |
| <b>MasVnrArea</b>   | 8           |
| <b>Electrical</b>   | 1           |

**dtype:** int64

```
threshold = len(housingData)*0.5
# drop those columns which have more than 50% nulls - based on what? how do we decide that?
threshold
```

730.0

```
housingDataClean = housingDataClean.dropna(thresh=threshold, axis=1)
housingDataClean.shape
```

(1460, 76)

```
# impute remaining missing values

# creating copy for comparison
hdc = housingDataClean.copy()

# 1. for numeric cols: fill with median
numericCols = hdc.select_dtypes(include=[np.number]).columns
# this will create dataframe numericCols, having only those columns in housingData that have n

for col in numericCols:
    hdc[col] = hdc[col].fillna(hdc[col].median())

# 2. For categorical columns: fill with mode
categoricalCols = hdc.select_dtypes(include=['object']).columns

for col in categoricalCols:
    hdc[col] = hdc[col].fillna(housingData[col].mode()[0])
```

```
# why is housingData used instead of hdc for finding the mode?
```

```
hdc.isnull().sum()
```

|                      | 0   |
|----------------------|-----|
| <b>Id</b>            | 0   |
| <b>MSSubClass</b>    | 0   |
| <b>MSZoning</b>      | 0   |
| <b>LotFrontage</b>   | 0   |
| <b>LotArea</b>       | 0   |
| ...                  | ... |
| <b>MoSold</b>        | 0   |
| <b>YrSold</b>        | 0   |
| <b>SaleType</b>      | 0   |
| <b>SaleCondition</b> | 0   |
| <b>SalePrice</b>     | 0   |

76 rows × 1 columns

**dtype:** int64

```
# one hot encoding - get_dummies
# This data, even though has more columns, has expanded each column into 0s and 1s, something
hdc = pd.get_dummies(hdc, drop_first=True)
hdc.shape
```

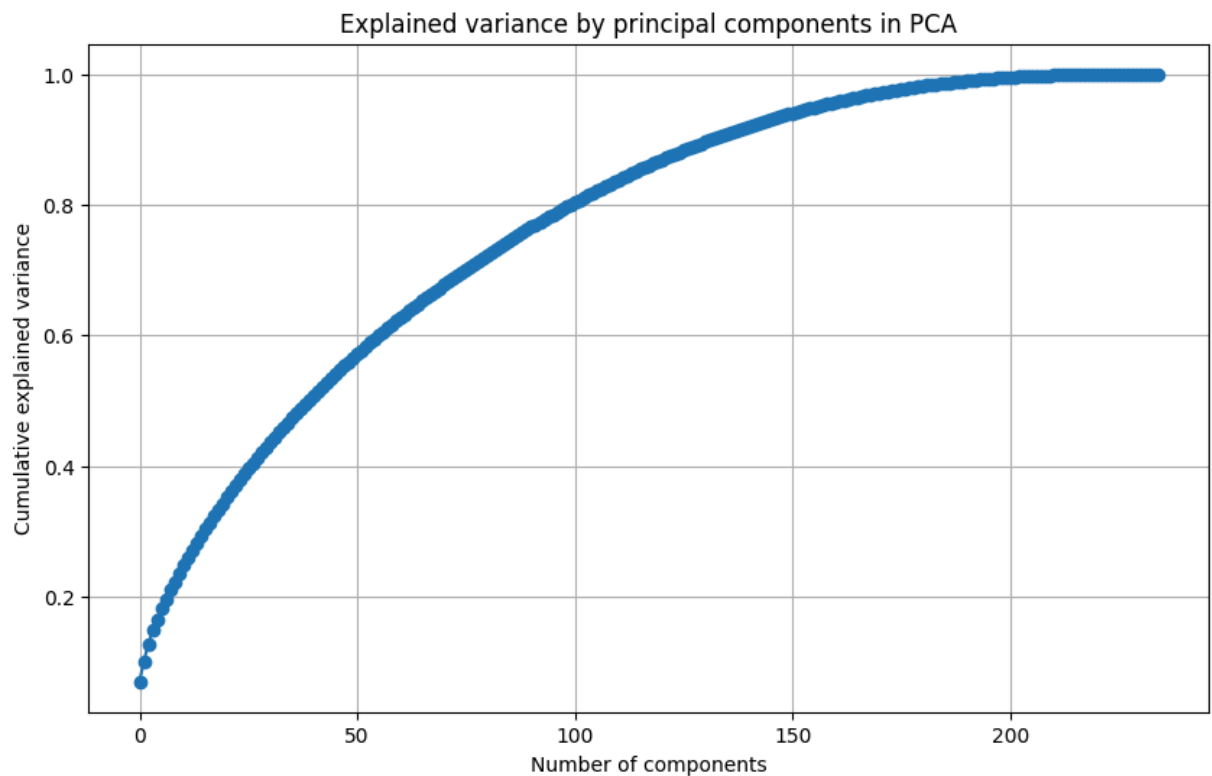
(1460, 235)

```
# before using PCA (principal component analysis), we need to standardize the data, and hence
# https://share.google/aimode/H2cTDiJgz2l00eD1D
scaler = StandardScaler()
scaledData = scaler.fit_transform(hdc)
print(type(scaler), type(scaledData), scaledData.shape)
```

<class 'sklearn.preprocessing.\_data.StandardScaler'> <class 'numpy.ndarray'> (1460, 235)

```
pca=PCA()
pca.fit(scaledData)

#plot explained variance
plt.figure(figsize=(10,6))
plt.plot(np.cumsum(pca.explained_variance_ratio_), marker='o')
plt.xlabel("Number of components")
plt.ylabel("Cumulative explained variance")
plt.title("Explained variance by principal components in PCA")
plt.grid(True)
plt.show()
```



```
pca = PCA(n_components=0.95) #what is n_components?
finalCleanedData = pca.fit_transform(scaledData)
print(f"Original shape: {housingData.shape}")
print(f"Shape after categorical variables and one hot encoding: {scaledData.shape}")
print(f"Reduced shape to 95%: {finalCleanedData.shape}")
```

Original shape: (1460, 81)  
 Shape after categorical variables and one hot encoding: (1460, 235)  
 Reduced shape to 95%: (1460, 157)

finalCleanedData

```
array([[ 4.27739044, -1.99772572, -1.29079801, ..., -0.38710153,
        -0.23980767, -0.13136502],
       [ 0.05292678,  3.40204193, -0.60065582, ...,  0.07807625,
        -0.44655168, -0.20959717],
       [ 5.01022523, -1.03432726, -0.57443954, ..., -0.20001591,
        -0.01235262, -0.22700231],
       ...,
       [ 0.85606202, -1.32662743,  3.28878399, ...,  0.81686647,
         0.24580904, -1.48603807],
       [-3.01281094,  3.45533125, -1.96329937, ...,  1.05024159,
         1.68355453,  0.31280536],
       [-1.60816723,  3.68277256, -1.81380672, ..., -0.6366799 ,
         0.3392235 ,  0.38787706]])
```

# Module 3: EDA on complex Datasets

```
# What does the leadership want?
# Overall statisitcs and what is the data saying mainly -> pandas.describe()
# Trends with data that they need to know, anything alarming -> pairplot
# Majorly represented neighborhoods -> countplot
```

```
pca_df = pd.DataFrame(finalCleanedData, columns=[f'PC{i+1}' for i in range(finalCleanedData.sh
```



```
pca_df['SalePrice'] = housingData['SalePrice'].values # bring back target for EDA
```

```
print ("Summary statistics of data before cleaning:")
print(housingData.describe(include='all'))
```

Summary statistics of data before cleaning:

|        | Id          | MSSubClass  | MSZoning | LotFrontage | LotArea       | Street | \ |
|--------|-------------|-------------|----------|-------------|---------------|--------|---|
| count  | 1460.000000 | 1460.000000 | 1460     | 1201.000000 | 1460.000000   | 1460   |   |
| unique | NaN         | NaN         | 5        | NaN         | NaN           | 2      |   |
| top    | NaN         | NaN         | RL       | NaN         | NaN           | Pave   |   |
| freq   | NaN         | NaN         | 1151     | NaN         | NaN           | 1454   |   |
| mean   | 730.500000  | 56.897260   | NaN      | 70.049958   | 10516.828082  | NaN    |   |
| std    | 421.610009  | 42.300571   | NaN      | 24.284752   | 9981.264932   | NaN    |   |
| min    | 1.000000    | 20.000000   | NaN      | 21.000000   | 1300.000000   | NaN    |   |
| 25%    | 365.750000  | 20.000000   | NaN      | 59.000000   | 7553.500000   | NaN    |   |
| 50%    | 730.500000  | 50.000000   | NaN      | 69.000000   | 9478.500000   | NaN    |   |
| 75%    | 1095.250000 | 70.000000   | NaN      | 80.000000   | 11601.500000  | NaN    |   |
| max    | 1460.000000 | 190.000000  | NaN      | 313.000000  | 215245.000000 | NaN    |   |

|        | Alley | LotShape | LandContour | Utilities | ... | PoolArea    | PoolQC | Fence | \ |
|--------|-------|----------|-------------|-----------|-----|-------------|--------|-------|---|
| count  | 91    | 1460     | 1460        | 1460      | ... | 1460.000000 | 7      | 281   |   |
| unique | 2     | 4        | 4           | 2         | ... | NaN         | 3      | 4     |   |
| top    | GrvL  | Reg      | Lvl         | AllPub    | ... | NaN         | Gd     | MnPrv |   |
| freq   | 50    | 925      | 1311        | 1459      | ... | NaN         | 3      | 157   |   |
| mean   | NaN   | NaN      | NaN         | NaN       | ... | 2.758904    | NaN    | NaN   |   |
| std    | NaN   | NaN      | NaN         | NaN       | ... | 40.177307   | NaN    | NaN   |   |
| min    | NaN   | NaN      | NaN         | NaN       | ... | 0.000000    | NaN    | NaN   |   |
| 25%    | NaN   | NaN      | NaN         | NaN       | ... | 0.000000    | NaN    | NaN   |   |
| 50%    | NaN   | NaN      | NaN         | NaN       | ... | 0.000000    | NaN    | NaN   |   |
| 75%    | NaN   | NaN      | NaN         | NaN       | ... | 0.000000    | NaN    | NaN   |   |
| max    | NaN   | NaN      | NaN         | NaN       | ... | 738.000000  | NaN    | NaN   |   |

|        | MiscFeature | MiscVal      | MoSold      | YrSold      | SaleType | \ |
|--------|-------------|--------------|-------------|-------------|----------|---|
| count  | 54          | 1460.000000  | 1460.000000 | 1460.000000 | 1460     |   |
| unique | 4           | NaN          | NaN         | NaN         | 9        |   |
| top    | Shed        | NaN          | NaN         | NaN         | WD       |   |
| freq   | 49          | NaN          | NaN         | NaN         | 1267     |   |
| mean   | NaN         | 43.489041    | 6.321918    | 2007.815753 | NaN      |   |
| std    | NaN         | 496.123024   | 2.703626    | 1.328095    | NaN      |   |
| min    | NaN         | 0.000000     | 1.000000    | 2006.000000 | NaN      |   |
| 25%    | NaN         | 0.000000     | 5.000000    | 2007.000000 | NaN      |   |
| 50%    | NaN         | 0.000000     | 6.000000    | 2008.000000 | NaN      |   |
| 75%    | NaN         | 0.000000     | 8.000000    | 2009.000000 | NaN      |   |
| max    | NaN         | 15500.000000 | 12.000000   | 2010.000000 | NaN      |   |

|        | SaleCondition | SalePrice     |
|--------|---------------|---------------|
| count  | 1460          | 1460.000000   |
| unique | 6             | NaN           |
| top    | Normal        | NaN           |
| freq   | 1198          | NaN           |
| mean   | NaN           | 180921.195890 |
| std    | NaN           | 79442.502883  |
| min    | NaN           | 34900.000000  |
| 25%    | NaN           | 129975.000000 |
| 50%    | NaN           | 163000.000000 |
| 75%    | NaN           | 214000.000000 |
| max    | NaN           | 755000.000000 |

[11 rows x 81 columns]

```
# summary statistics after cleaning
print ("Summary statistics of data after cleaning:")
print(hdc.describe(include='all'))
```

Summary statistics of data after cleaning:

|        | Id          | MSSubClass  | LotFrontage | LotArea     | OverallQual | \ |
|--------|-------------|-------------|-------------|-------------|-------------|---|
| count  | 1460.000000 | 1460.000000 | 1460.000000 | 1460.000000 | 1460.000000 |   |
| unique | NaN         | NaN         | NaN         | NaN         | NaN         |   |
| top    | NaN         | NaN         | NaN         | NaN         | NaN         |   |

|      |             |            |            |               |           |
|------|-------------|------------|------------|---------------|-----------|
| freq | NaN         | NaN        | NaN        | NaN           | NaN       |
| mean | 730.500000  | 56.897260  | 69.863699  | 10516.828082  | 6.099315  |
| std  | 421.610009  | 42.300571  | 22.027677  | 9981.264932   | 1.382997  |
| min  | 1.000000    | 20.000000  | 21.000000  | 1300.000000   | 1.000000  |
| 25%  | 365.750000  | 20.000000  | 60.000000  | 7553.500000   | 5.000000  |
| 50%  | 730.500000  | 50.000000  | 69.000000  | 9478.500000   | 6.000000  |
| 75%  | 1095.250000 | 70.000000  | 79.000000  | 11601.500000  | 7.000000  |
| max  | 1460.000000 | 190.000000 | 313.000000 | 215245.000000 | 10.000000 |

|        |             |             |              |             |             |     |   |
|--------|-------------|-------------|--------------|-------------|-------------|-----|---|
|        | OverallCond | YearBuilt   | YearRemodAdd | MasVnrArea  | BsmtFinSF1  | ... | \ |
| count  | 1460.000000 | 1460.000000 | 1460.000000  | 1460.000000 | 1460.000000 | ... |   |
| unique | NaN         | NaN         | NaN          | NaN         | NaN         | ... |   |
| top    | NaN         | NaN         | NaN          | NaN         | NaN         | ... |   |
| freq   | NaN         | NaN         | NaN          | NaN         | NaN         | ... |   |
| mean   | 5.575342    | 1971.267808 | 1984.865753  | 103.117123  | 443.639726  | ... |   |
| std    | 1.112799    | 30.202904   | 20.645407    | 180.731373  | 456.098091  | ... |   |
| min    | 1.000000    | 1872.000000 | 1950.000000  | 0.000000    | 0.000000    | ... |   |
| 25%    | 5.000000    | 1954.000000 | 1967.000000  | 0.000000    | 0.000000    | ... |   |
| 50%    | 5.000000    | 1973.000000 | 1994.000000  | 0.000000    | 383.500000  | ... |   |
| 75%    | 6.000000    | 2000.000000 | 2004.000000  | 164.250000  | 712.250000  | ... |   |
| max    | 9.000000    | 2010.000000 | 2010.000000  | 1600.000000 | 5644.000000 | ... |   |

|        |                |                |              |              |   |
|--------|----------------|----------------|--------------|--------------|---|
|        | SaleType_ConLI | SaleType_ConLw | SaleType_New | SaleType_Oth | \ |
| count  | 1460           | 1460           | 1460         | 1460         |   |
| unique | 2              | 2              | 2            | 2            |   |
| top    | False          | False          | False        | False        |   |
| freq   | 1455           | 1455           | 1338         | 1457         |   |
| mean   | NaN            | NaN            | NaN          | NaN          |   |
| std    | NaN            | NaN            | NaN          | NaN          |   |
| min    | NaN            | NaN            | NaN          | NaN          |   |
| 25%    | NaN            | NaN            | NaN          | NaN          |   |
| 50%    | NaN            | NaN            | NaN          | NaN          |   |
| 75%    | NaN            | NaN            | NaN          | NaN          |   |
| max    | NaN            | NaN            | NaN          | NaN          |   |

|        |             |                       |                      |   |
|--------|-------------|-----------------------|----------------------|---|
|        | SaleType_WD | SaleCondition_AdjLand | SaleCondition_Alloca | \ |
| count  | 1460        | 1460                  | 1460                 |   |
| unique | 2           | 2                     | 2                    |   |
| top    | True        | False                 | False                |   |
| freq   | 1267        | 1456                  | 1448                 |   |
| mean   | NaN         | NaN                   | NaN                  |   |
| std    | NaN         | NaN                   | NaN                  |   |
| min    | NaN         | NaN                   | NaN                  |   |
| 25%    | NaN         | NaN                   | NaN                  |   |
| 50%    | NaN         | NaN                   | NaN                  |   |
| 75%    | NaN         | NaN                   | NaN                  |   |
| max    | NaN         | NaN                   | NaN                  |   |

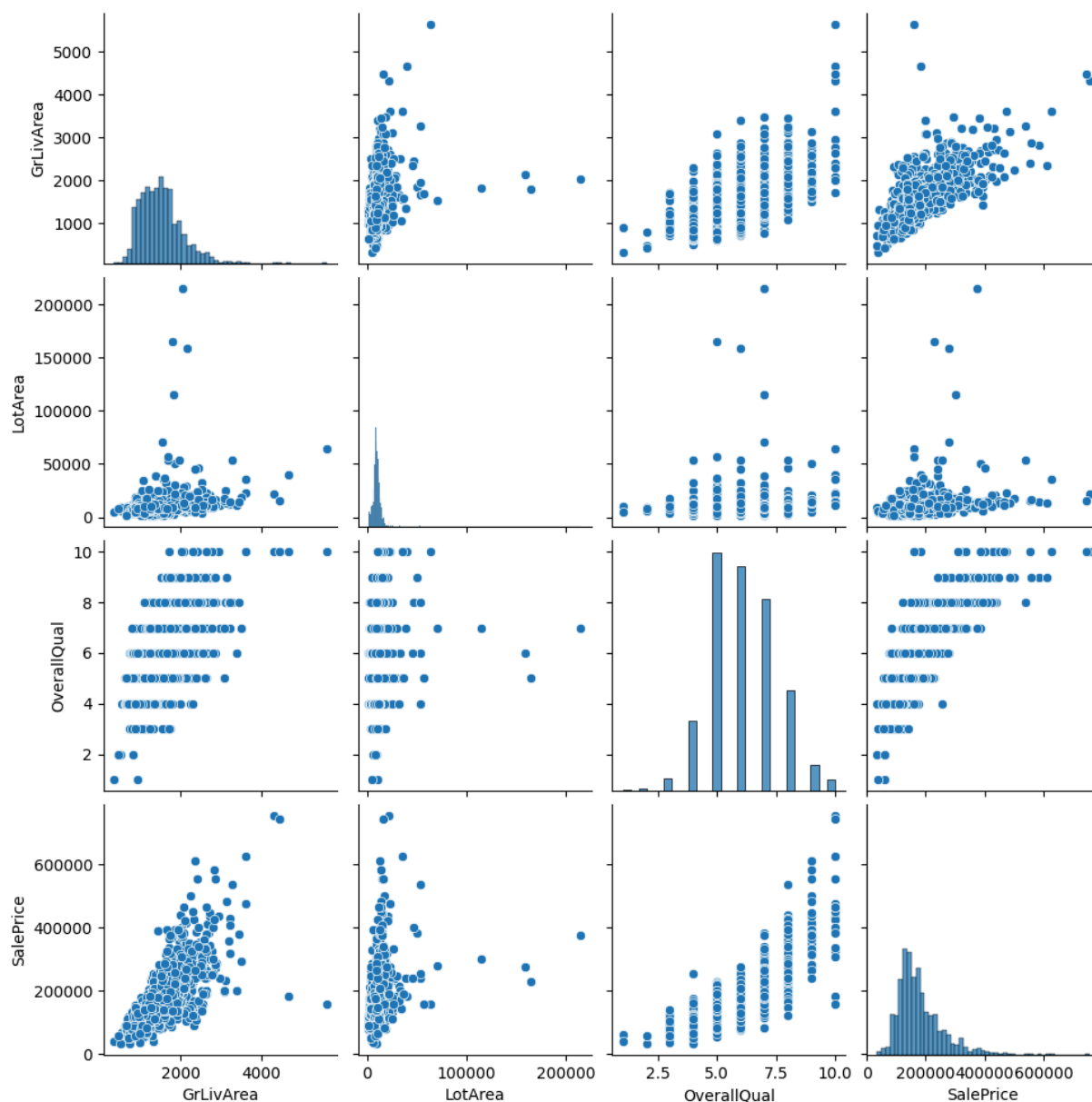
  

|        |                      |                      |                       |
|--------|----------------------|----------------------|-----------------------|
|        | SaleCondition_Family | SaleCondition_Normal | SaleCondition_Partial |
| count  | 1460                 | 1460                 | 1460                  |
| unique | 2                    | 2                    | 2                     |
| top    | False                | True                 | False                 |

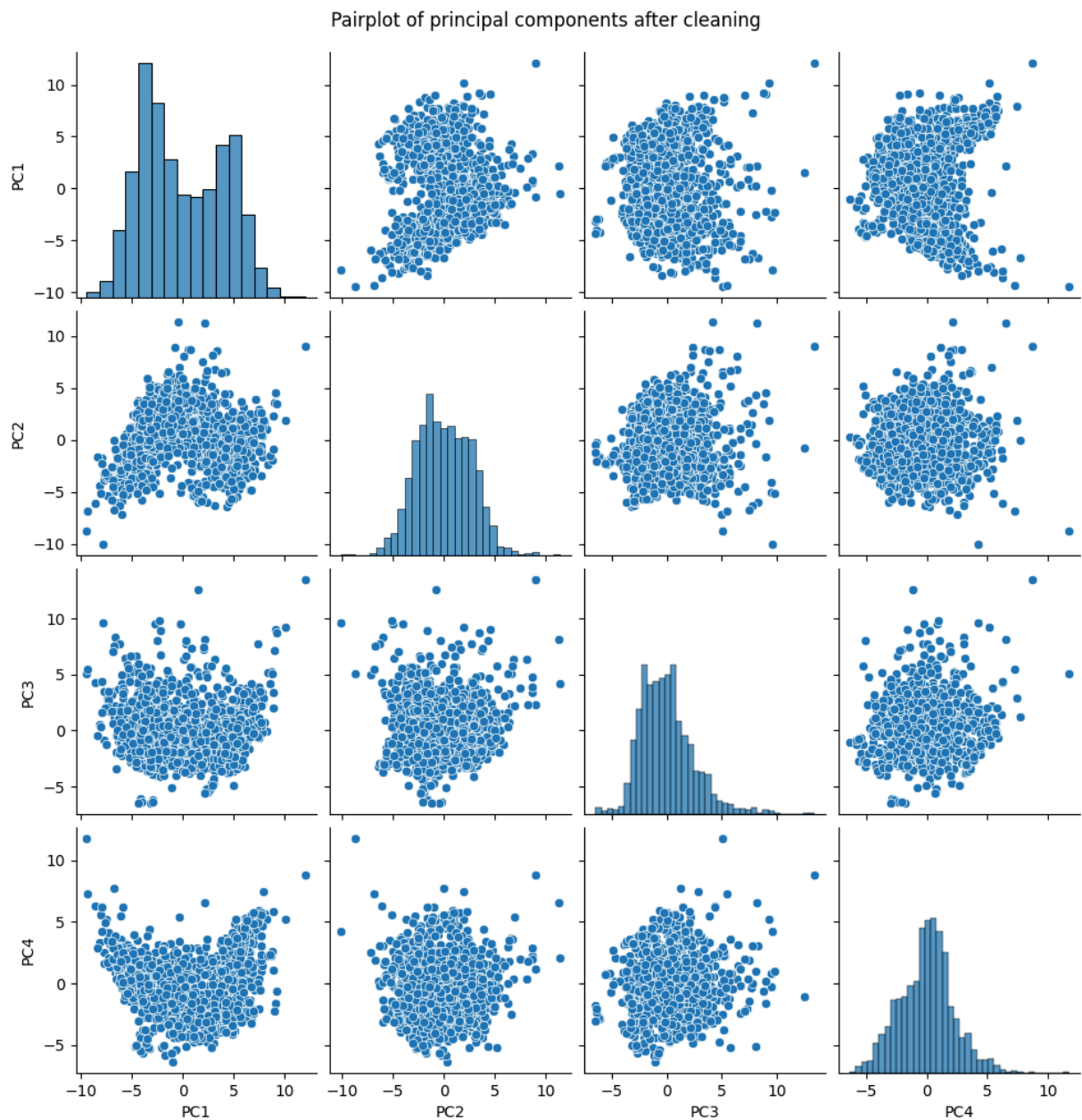
```
# pairplot before cleaning
sns.pairplot(housingData[['GrLivArea', 'LotArea', 'OverallQual', 'SalePrice']])
plt.suptitle("Pairplot features before cleaning", y=1.02)
```

Text(0.5, 1.02, 'Pairplot features before cleaning')

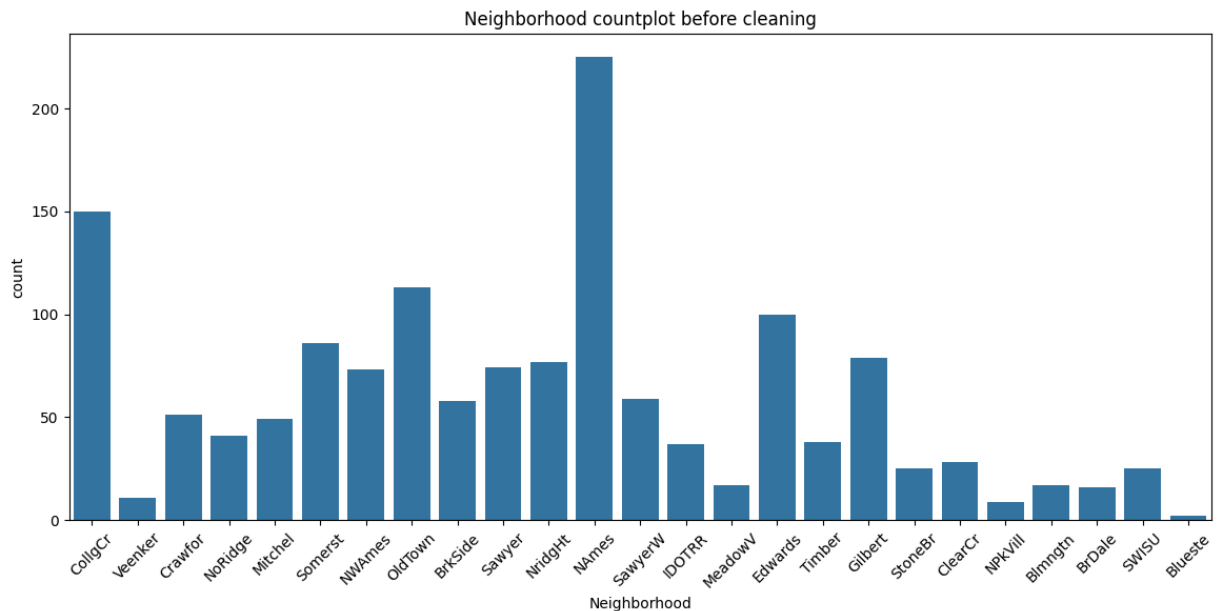
Pairplot features before cleaning



```
sns.pairplot(pca_df[['PC1', 'PC2', 'PC3', 'PC4']])
plt.suptitle("Pairplot of principal components after cleaning", y=1.02)
plt.show()
```



```
# countplot: histogram for visualizing any data with high cardinality to visualize the spread
plt.figure(figsize=(14,6))
sns.countplot(x='Neighborhood', data=housingData)
plt.xticks(rotation=45)
plt.title("Neighborhood countplot before cleaning")
plt.show()
```



```
neighboroodReplacements = {
    'Blmngtn' : 'Bloomington',
    'Blueste' : 'Bluestem',
    'BrDale' : 'Briardale',
    'BrkSide' : 'Brookside',
    'ClearCr' : 'Clear Creek',
    'CollgCr' : 'College Creek',
    'Crawfor' : 'Crowford',
    'Edwards' : 'Edwards',
    'Gilbert' : 'Gilbert',
    'IDOTRR' : 'IDOTRR',
    'MeadowV' : 'Meadow Village',
    'Mitchel' : 'Mitchell',
    'NAMES' : 'Names',
    'NoRidge' : 'North Ridge',
    'NPkVill' : 'Npk Villa',
    'NridgHt' : 'Northridge Heights',
    'NWAmes' : 'Northwest Ames',
    'OldTown' : 'Old Town',
    'Sawyer' : 'Sawyer',
    'SawyerW' : 'Sawyer West',
    'Somerst' : 'Somerset',
    'StoneBr' : 'Store Bridge',
    'SWISU' : 'Swisu',
    'Timber' : 'Timber'
}

housingData['NeighborhoodCleaned'] = housingData['Neighborhood'].replace(neighboroodReplacements)
housingData.shape
```

```
(1460, 82)
```

```
# use cleaned names of the neighborhood for the plot
nghb_counts_cleaned = housingData['NeighborhoodCleaned'].value_counts()
nghb_counts_cleaned
```

|                     | count |
|---------------------|-------|
| NeighborhoodCleaned |       |
| Names               | 225   |
| College Creek       | 150   |
| Old Town            | 113   |
| Edwards             | 100   |
| Somerset            | 86    |
| Gilbert             | 79    |
| Northridge Heights  | 77    |
| Sawyer              | 74    |
| Northwest Ames      | 73    |
| Sawyer West         | 59    |
| Brookside           | 58    |
| Crowford            | 51    |
| Mitchell            | 49    |
| North Ridge         | 41    |
| Timber              | 38    |
| IDOTRR              | 37    |
| Clear Creek         | 28    |
| Swisu               | 25    |
| Store Bridge        | 25    |
| Bloomington         | 17    |
| Meadow Village      | 17    |
| Briardale           | 16    |
| Veenker             | 11    |
| Npk Villa           | 9     |
| Bluestem            | 2     |

**dtype:** int64

```
top_nghb_cleaned = nghb_counts_cleaned[nghb_counts_cleaned > 50].index
top_nghb_cleaned
```

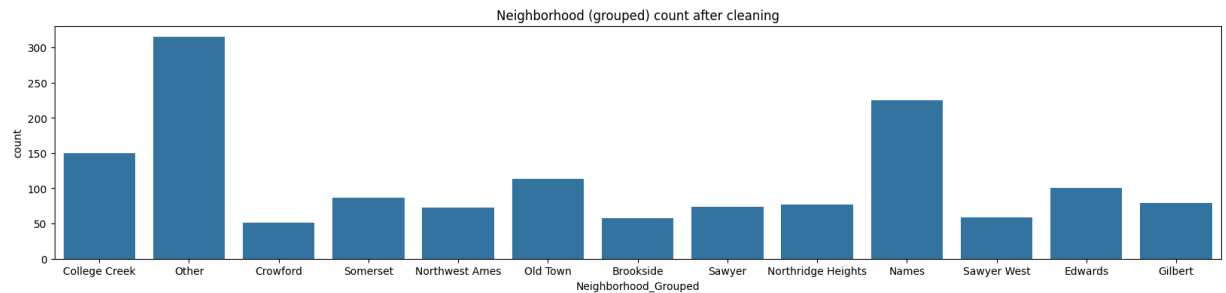
```
Index(['Names', 'College Creek', 'Old Town', 'Edwards', 'Somerset', 'Gilbert',
      'Northridge Heights', 'Sawyer', 'Northwest Ames', 'Sawyer West',
      'Brookside', 'Crowford'],
      dtype='object', name='NeighborhoodCleaned')
```

```
hdc['Neighborhood_Grouped'] = housingData['NeighborhoodCleaned'].apply(
    lambda x: x if x in top_nghb_cleaned else 'Other'
)
```

```
hdc.shape
```

```
(1460, 236)
```

```
plt.figure(figsize=(20,4))
sns.countplot(x='Neighborhood_Grouped', data=hdc)
plt.title("Neighborhood (grouped) count after cleaning")
plt.show()
```



```
from sklearn.cluster import KMeans
```

```
sampleDf = hdc.sample(frac=0.1, random_state=42) #10% sample rate
```

```
features = ['GrLivArea', 'LotArea', 'OverallQual']
xScaled = scaler.fit_transform(sampleDf[features])
```

```
import multiprocessing, os
try:
    print (f"Physical Cores: {len(os.sched_getaffinity(0))}")
except AttributeError:
    print (f"Physical Cores: {multiprocessing.cpu_count()}")
```

Physical Cores: 2

```
inertia = []
k_range = range(1,8)
for k in k_range:
    kmeans = KMeans(n_clusters=k, random_state=42)
    kmeans.fit(xScaled)
    inertia.append(kmeans.inertia_)
```

```
# plot Elbow curve
plt.figure(figsize=(8,5))
plt.plot(k_range, inertia, marker='o')
plt.title("Elbow Method for optimal K")
plt.xlabel("Number of clusters")
plt.ylabel("Inertia")
plt.grid(True)
plt.show()
```

